

Lista de Revisão 2

Orientações para resolução da lista

- Resolva os exercícios utilizando exclusivamente os conceitos abordados em aula até o momento (variáveis, expressões, estruturas condicionais e de repetição).
- Antes de implementar, leia atentamente o enunciado e procure identificar claramente:
 - quais são as entradas do problema;
 - qual processamento deve ser realizado;
 - qual saída é esperada.
- Sempre que possível, elabore casos de teste antes da implementação. Teste situações típicas, limites e casos extremos.
- Evite começar diretamente pelo código. Pense na lógica da solução e, se necessário, descreva os passos em linguagem natural ou pseudocódigo.
- Utilize nomes de variáveis significativos, que representem claramente o papel de cada valor no problema.
- Em exercícios com entrada contínua, tenha atenção especial às condições de parada (sentinelas), como valores que indicam o fim da leitura.
- Em estruturas de repetição, verifique cuidadosamente:
 - a condição de parada;
 - a atualização das variáveis de controle;
 - possíveis situações de loop infinito.
- Em problemas que envolvem decisões, analise se as condições são independentes ou se devem ser mutuamente exclusivas.
- Sempre valide seu programa com diferentes conjuntos de entrada para garantir que o comportamento está correto.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver sua autonomia na resolução de problemas.

1. Você está desenvolvendo o módulo de segurança de um aplicativo financeiro. Durante o cadastro, o sistema pede ao usuário que crie uma senha. Crie uma função `validar_senha(senha)` que verifique se a senha é segura. Uma senha é válida se: tiver pelo menos 8 caracteres, contiver pelo menos um número, contiver pelo menos uma letra maiúscula e não contiver espaços. O programa principal deve ler senhas repetidamente até que o usuário digite "sair", imprimindo "Válida" ou "Inválida" para cada tentativa, e ao final informar a porcentagem de senhas válidas testadas.
2. Um mapa de jogo é representado por uma matriz J de dimensão $N \times M$ onde cada célula possui uma pontuação. O jogador começa em $J[0][0]$ e só pode se mover para a direita ou para baixo. Escreva uma função que calcule a maior pontuação possível que o jogador pode coletar ao chegar em $J[N - 1][M - 1]$.
3. Crie um programa que leia 20 números inteiros e os armazene em uma lista. Crie uma função `eliminar_duplicatas(lista)` que retorne uma nova lista contendo os mesmos elementos, mas mantendo apenas a primeira ocorrência de cada número (preservando a ordem original). Observação: Não utilize conjuntos (*sets*) para resolver a questão.
4. Um sistema de biblioteca utiliza um dicionário onde a `chave` é o ISBN do livro e o valor é uma lista `[Titulo, Autor, Quantidade_Estoque]`. Crie funções para `emprestar_livro(isbn)` (reduz estoque ou avisa se não há) e `devolver_livro(isbn)`, que aumenta o estoque do livro devolvido.
5. Uma provedora de nuvem cobra por hora de uso de instâncias de servidor. Crie uma função `calcular_fatura(horas)` que aplique as seguintes regras: as primeiras 100 horas custam R\$ 5,00/hora; de 101 a 500 horas, o custo cai para R\$ 4,00/hora para as horas excedentes; acima de 500 horas, o custo cai para R\$ 2,50/hora para as horas excedentes. Faça um programa que leia o ID do cliente e horas consumidas até que o ID seja 0. Imprima a fatura de cada cliente e o faturamento total da empresa.
6. Uma plataforma acadêmica possui a lista de alunos matriculados em "Algoritmos" e a lista de alunos matriculados em "Cálculo". O programa deve ler N matrículas para cada disciplina. Utilizando a estrutura de Conjuntos (*set*), determine e imprima: os alunos matriculados em ambas as disciplinas, e os alunos matriculados em apenas uma das disciplinas.
7. Em análise de imagens, um filtro de suavização substitui o valor de um pixel pela média dele e de seus vizinhos diretos (cima, baixo, esquerda, direita). Dada uma matriz de inteiros $N \times N$, aplique esse filtro a todos os elementos internos (excluindo as bordas) e imprima a nova matriz.
8. O sistema de rastreamento de entregas armazena o percurso de um caminhão como uma lista de tuplas (`latitude, longitude`). Escreva uma função que receba essa lista e calcule a distância total percorrida. Para simplificar, considere a distância entre os pontos (x_1, y_1) e (x_2, y_2) como $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

9. Uma cooperativa agrícola possui um silo com capacidade limitada de sementes. Crie um procedimento `processar_pedidos(estoque_inicial)` que leia continuamente o peso desejado pelos agricultores. Se o valor for maior que o estoque, o pedido é negado. Caso contrário, o estoque é reduzido. O procedimento deve encerrar quando o estoque zerar ou quando um pedido negativo for inserido, exibindo o total de agricultores atendidos e a sobra no silo.
10. Desenvolva um contador de frequência de palavras. O programa lê um parágrafo de texto, divide em palavras, converte tudo para minúsculas e remove pontuações. Utilize um dicionário (em que cada chave representa uma palavra encontrada no texto, e cada valor representa a quantidade de vezes que essa palavra apareceu) para contar quantas vezes cada palavra apareceu e, ao final, imprima as top 3 palavras mais frequentes.
11. Um drone de entregas precisa validar suas condições antes de decolar. Crie uma função `autorizar_voo(bateria, velocidade_vento)` que retorne `True` se a bateria for maior que 15% e o vento for menor que 30 km/h. O programa principal deve ler os dados de 5 etapas de checagem. Em cada etapa, a bateria cai 2%. Se o voo for proibido em qualquer etapa, a função deve acionar um alerta e o programa deve abortar a missão, indicando em qual etapa ocorreu a falha.
12. Desenvolva um programa que leia uma matriz de dimensões $M \times N$ e gere a sua matriz transposta ($N \times M$). Crie uma função específica `transpor(M)` para realizar esta operação, retornando a nova estrutura sem alterar a original.
13. Escreva uma função que receba uma tupla contendo notas de jurados em uma competição de ginástica. A função deve descartar a maior e a menor nota e retornar a média das notas restantes. Lembre-se que tuplas são imutáveis. Na função principal, faça um código que leia as notas e chame a função desenvolvida.
14. Faça uma função `calcular_bonus(pontos)` para um sistema de RH. Se a pontuação for maior que 50, o bônus é de R\$ 100,00; caso contrário, R\$ 10,00. Em seguida, crie um programa que leia a pontuação de N funcionários, armazene-os em uma lista e use a função para substituir a pontuação na lista pelo valor do bônus recebido, imprimindo a lista final e o total pago.
15. Peça ao usuário que informe uma sequência de números inteiros, parando ao digitar 0. Armazene os números em uma lista. Desenvolva uma função que receba essa lista e determine o tamanho do maior trecho crescente consecutivo presente.
16. Uma loja deseja analisar os preços utilizando um dicionário. Crie um programa que armazene produtos e preços. A leitura para quando o produto for "fim". Calcule a média dos valores e crie uma função que receba o dicionário e retorne uma tupla com: o produto mais caro, o mais barato e uma lista com produtos acima da média.
17. Crie uma função `matriz_simetrica(M)` que receba uma matriz quadrada $N \times N$ e retorne `True` se ela for simétrica em relação à diagonal principal. Para que uma matriz seja simétrica, $M[i][j] == M[j][i]$ para todos os pares i, j .

18. Um sistema de análise de desempenho de entregadores registra o tempo previsto e o tempo gasto em minutos. Leia N entregas e armazene-as em duas listas (previsto e real). Crie uma função que receba essas listas e retorne uma nova lista contendo as classificações: "Adiantada", "No prazo" ou "Atrasada". O programa deve calcular o atraso médio apenas das entregas atrasadas.
19. Uma análise de rede social identificou as tags utilizadas por **um único usuário** em diferentes publicações. Cada publicação possui uma lista de tags, representadas por strings. Como uma mesma tag pode aparecer em várias publicações, ou até repetida dentro de uma mesma lista, deseja-se contar apenas os assuntos diferentes abordados por esse usuário. Desenvolva uma função chamada `tags_unicas(listas_de_tags)` que receba uma lista contendo várias listas de tags. A função deve usar conjuntos para reunir todas as tags distintas presentes em todas as publicações desse usuário e retornar o total de tags únicas encontradas. Em outras palavras, a função deve considerar todas as listas de tags como pertencentes ao mesmo usuário e contar quantas tags diferentes aparecem no conjunto total de publicações.
20. Considere uma lista de filmes chamada `filmes`. Cada filme é representado por um dicionário com duas informações:
- "titulo": uma string com o título do filme;
 - "generos": uma lista de strings com os gêneros aos quais o filme pertence.

A estrutura geral da lista será semelhante a:

```
filmes = [  
    {"titulo": "Filme A", "generos": ["Ação", "Sci-Fi"]},  
    {"titulo": "Filme B", "generos": ["Drama"]},  
    {"titulo": "Filme C", "generos": ["Ação", "Aventura"]}  
]
```

Desenvolva uma função chamada `catalogo_por_genero(filmes)` que receba essa lista e crie uma nova estrutura organizada por gênero.

A função deve retornar um dicionário em que:

- cada chave seja um gênero encontrado nos filmes;
- cada valor seja uma lista contendo os títulos dos filmes que pertencem àquele gênero.

Um mesmo filme pode aparecer em mais de um gênero, caso sua lista de gêneros tenha mais de um item.

21. O sistema de uma universidade controla o acesso dos alunos a um laboratório especial. Para entrar no laboratório, cada aluno deve atender a algumas regras relacionadas à idade, ao treinamento obrigatório e à autorização do responsável.
- Crie uma função chamada `acesso_liberado(idade, treinamento, autorizacao)` que receba:

- `idade`: idade do aluno;
- `treinamento`: valor booleano indicando se o aluno realizou o treinamento obrigatório;
- `autorizacao`: valor booleano indicando se o aluno possui autorização do responsável.

A função deve retornar a string "Permitido" ou "Negado", de acordo com as seguintes regras:

- alunos com 18 anos ou mais podem entrar apenas se tiverem realizado o treinamento obrigatório;
- alunos menores de 18 anos podem entrar apenas se tiverem realizado o treinamento obrigatório e possuírem autorização do responsável;
- alunos sem treinamento obrigatório nunca podem entrar, independentemente da idade ou da autorização.

Depois, crie um programa principal que leia os dados de vários alunos. Para cada aluno, devem ser lidas as seguintes informações:

- nome do aluno;
- idade;
- se realizou o treinamento obrigatório;
- se possui autorização do responsável.

A leitura deve continuar até que seja informada a idade 0. Quando isso acontecer, o programa deve encerrar a leitura e não deve cadastrar esse aluno.

Para cada aluno lido, o programa deve utilizar a função `acesso_liberado` para verificar se o acesso será permitido ou negado.

Ao final, o programa deve criar um dicionário chamado `relatorio_acessos`, no seguinte formato:

```
{
    "Permitido": ["Ana", "Carlos"],
    "Negado": ["Bruno", "Marina"]
}
```

Nesse dicionário:

- a chave "Permitido" deve armazenar uma lista com os nomes dos alunos que tiveram acesso liberado;
- a chave "Negado" deve armazenar uma lista com os nomes dos alunos que tiveram acesso negado.

Além disso, ao final, o programa deve exibir:

- o dicionário `relatorio_acessos`;

- a quantidade total de alunos com acesso permitido;
 - a quantidade total de alunos com acesso negado;
 - uma mensagem indicando se a maioria dos alunos teve acesso permitido, acesso negado ou se houve empate.
- 22.** Crie uma função que receba três números reais X, Y e Z , verifique se podem formar um triângulo (cada lado menor que a soma dos outros dois) e retorne o tipo: Equilátero, Isósceles ou Escaleno. O programa deve ler N triângulos e exibir a contagem de cada tipo formado.
- 23.** Escreva uma função que receba uma lista de números reais e um valor limite L . A função deve remover da lista todos os elementos que sejam menores que L , ajustando o tamanho da lista, e retornar a quantidade de elementos removidos.
- 24.** Verifique se uma matriz $N \times N$ fornecida pelo usuário é um "Quadrado Mágico". Uma matriz é mágica se a soma dos elementos de cada linha, de cada coluna e das duas diagonais resulta no mesmo valor, e contém todos os números de 1 a N^2 . Faça um programa que receba o valor de N , depois os valores da matriz, e diga se ela é ou não um quadrado mágico.

Exemplo de entrada correspondente a um quadrado mágico:

```
3
8 1 6
3 5 7
4 9 2
```

- 25.** Desenvolva um programa de votação. Leia uma quantidade indeterminada de tuplas contendo (`ID_Eleitor`, `Candidato`). Um eleitor não pode votar duas vezes (o voto mais recente é o que vale). Utilize estruturas adequadas (como conjuntos para rastrear IDs e dicionários para os votos) para computar a apuração final.
- 26.** Desenvolva um programa que leia os preços de N produtos em um carrinho de compras. Crie uma função que aplique um desconto progressivo na lista: 5% para itens acima de R\$ 100, e 10% para itens acima de R\$ 500. A função deve retornar a lista de preços atualizada e o valor total economizado.
- 27.** Crie um sistema de reservas de poltronas de um teatro representado por uma matriz 10×10 , inicializada com 0 (livre). O usuário informa linha e coluna para reservar. Crie funções para: `reservar(M, l, c)`, que marca com 1 o local, se esse local estiver livre; e `verificar_lotacao(M)`, que retorna a porcentagem de ocupação total do teatro.
- 28.** Crie uma função que receba duas listas ordenadas de inteiros (tamanhos diferentes) e retorne uma terceira lista, também ordenada, contendo a mescla dos elementos das duas listas, sem usar a função `sort()` do Python.

29. Um sensor de temperatura realiza medições e as guarda em uma lista. Sabendo que o sensor às vezes falha e registra -999, crie uma função que receba a lista de medições e substitua todos os -999 pela média aritmética dos dois valores válidos adjacentes (o anterior e o próximo).
30. Uma empresa deseja criar um programa para analisar os dados de seus funcionários. Cada funcionário deve ser armazenado em uma lista de dicionários, em que cada dicionário representa um funcionário e possui as seguintes chaves:
- "nome": nome do funcionário;
 - "setor": setor em que o funcionário trabalha;
 - "cargo": cargo ocupado pelo funcionário;
 - "salario": salário mensal do funcionário;
 - "tempo_empresa": tempo de empresa, em anos.

Crie um programa em Python que leia os dados de vários funcionários. A leitura deve continuar até que o nome informado seja "fim".

Durante a leitura, o programa deve validar os dados informados:

- o nome não pode ser vazio;
- o setor não pode ser vazio;
- o cargo não pode ser vazio;
- o salário deve ser maior que zero;
- o tempo de empresa deve ser maior ou igual a zero.

Funcionários com dados inválidos não devem ser adicionados à lista.

Após a leitura dos dados, crie uma função chamada `analisar_setores(funcionarios)`. Essa função deve receber a lista de funcionários e retornar um dicionário em que cada chave seja o nome de um setor da empresa.

Para cada setor, o valor associado deve ser outro dicionário contendo as seguintes informações:

- "quantidade": quantidade de funcionários do setor;
- "media_salarial": média salarial dos funcionários do setor;
- "maior_salario": maior salário encontrado no setor;
- "menor_salario": menor salário encontrado no setor;
- "funcionario_maior_salario": nome do funcionário com maior salário no setor;
- "media_tempo_empresa": média do tempo de empresa dos funcionários do setor.

Além disso, crie uma segunda função chamada `classificar_setores(relatorio)`. Essa função deve receber o dicionário gerado pela função `analisar_setores` e retornar uma lista com os nomes dos setores que estão com a média salarial maior que a média salarial **geral** da empresa.

Ao final, o programa deve exibir:

- a lista completa de funcionários cadastrados;
- o dicionário de análise por setor;
- os setores que tem sua média salarial acima da média salarial geral;
- o setor com a maior média salarial;
- o setor com a menor média salarial;
- uma mensagem indicando se existe algum setor com apenas um funcionário.

Caso nenhum funcionário válido tenha sido cadastrado, o programa deve exibir uma mensagem informando que não há dados suficientes para gerar o relatório.

Questões de Múltipla Escolha

31. Em sistemas computacionais, o conceito de escopo de variáveis e passagem de parâmetros define como os dados são gerenciados em memória. Considere o trecho de código abaixo:

```
1 usuario_ativo = False
2 tentativas = 0
3
4 def atualizar_status(tentativas):
5     usuario_ativo = True
6     tentativas = tentativas + 1
7     return tentativas
8
9 tentativas_restantes = atualizar_status(tentativas)
```

Com base no comportamento desse algoritmo, avalie as assertivas a seguir:

- I. A instrução `usuario_ativo = True` dentro da função altera o valor da variável global `usuario_ativo` para `True`.
- II. A variável `tentativas` definida na assinatura da função e a variável `tentativas` definida fora da função ocupam espaços de memória distintos.
- III. O valor final armazenado na variável global `tentativas_restantes` será 1, enquanto a variável global `tentativas` permanecerá com o valor 0.

É correto o que se afirma em:

- A) Apenas I e II.
- B) Apenas I e III.
- C) Apenas II e III.
- D) Apenas II.
- E) I, II e III.

32. Avalie as asserções a seguir e a relação proposta entre elas:

- I. Ao passar uma estrutura de lista para uma função em Python e alterar seus índices internamente (ex: `lista_parametro[0] = 99`), a lista original no escopo global será permanentemente alterada sem a necessidade de uso do comando `return`.

PORQUE

- II. Em Python, as estruturas de dados compostas, como listas e dicionários, são mutáveis e suas referências em memória são compartilhadas entre o escopo global e local durante a passagem de parâmetros.

Assinale a opção correta:

- A) As assertões I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B) As assertões I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C) A assertão I é uma proposição verdadeira, e a II é uma proposição falsa.
- D) A assertão I é uma proposição falsa, e a II é uma proposição verdadeira.
- E) As assertões I e II são proposições falsas.

33. Considere o algoritmo a seguir, projetado para operar sobre uma matriz quadrada M de dimensões $N \times N$:

```
1 for i in range(N):
2     for j in range(i + 1, N):
3         aux = M[i][j]
4         M[i][j] = M[j][i]
5         M[j][i] = aux
```

Conclui-se que o objetivo principal desse algoritmo é:

- A) Ordenar os elementos de cada linha da matriz em ordem crescente.
- B) Zerar os elementos localizados abaixo da diagonal principal da matriz.
- C) Espelhar a matriz horizontalmente, invertendo a ordem das colunas.
- D) Transpor a matriz in-place, trocando os elementos de linhas por colunas.
- E) Somar os elementos correspondentes da diagonal principal e secundária.

34. Avalie os seguintes cenários hipotéticos de Engenharia de Software:

- I. Para modelar a imutabilidade das coordenadas geográficas fixas de hospitais no sistema, deve-se utilizar a estrutura de **Listas**.
- II. Para garantir que em uma roleta nenhum participante seja cadastrado pelo seu CPF mais de uma vez, a estrutura ideal é o **Conjunto (Set)**.
- III. Para modelar os metadados de uma requisição web, onde cada identificador aponta para um valor heterogêneo, a estrutura adequada é o **Dicionário (Dict)**.

É correto o que se afirma em:

- A) Apenas I e II.

- B) Apenas I e III.
- C) Apenas II e III.
- D) Apenas I.
- E) I, II e III.

35. Considere a validação de acesso descrita na expressão a seguir:

```
1 if usuario_logado and bd.consultar_privilegios() == "ADMIN":  
2     liberar_painel()
```

Avalie as asserções a seguir:

- I. Se a variável `usuario_logado` for `False`, a função `bd.consultar_privilegios()` não será executada.

PORQUE

- II. Na avaliação do operador lógico `and`, caso o operando à esquerda seja resolvido como falso, a linguagem garante que o resultado global será falso, suprimindo a execução das expressões à direita.

Assinale a opção correta:

- A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
 - B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
 - C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
 - D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
 - E) As asserções I e II são proposições falsas.
36. O conceito pilar do Pensamento Computacional que consiste em dividir um problema grande e complexo em partes menores, coesas e independentes chama-se:
- A) Algoritmo Recursivo.
 - B) Abstração de Dados.
 - C) Decomposição.
 - D) Avaliação de Curto-Circuito.
 - E) Estruturação Bidimensional.

37. Considere um programa desenvolvido para iterar sobre dicionários aninhados:

```
1 vendas = {  
2     "V01": {"cliente": "Ana", "valor": 500.00},  
3     "V02": {"cliente": "Mateus", "valor": 1200.00}  
4 }  
5 destaque = ""  
6 maior = 0  
7 for cod, dados in vendas.items():  
8     if dados["valor"] > maior:  
9         maior = dados["valor"]  
10        destaque = dados["cliente"]  
11 print(destaque)
```

Qual a saída deste bloco de código?

- A) 1200.00
 - B) O código lançará um erro de sintaxe por tentar acessar dados["valor"] sem índices numéricos.
 - C) V02
 - D) Mateus
 - E) "Mateus": 1200.00
38. Ao manipular laços while e for para percorrer Listas e Matrizes, avalie as assertivas:
- I. A construção for linha in M: itera sobre cada elemento numérico individual de M de forma linear.
 - II. Para alterar o valor interno de uma célula e refletir essa alteração na estrutura global, deve-se realizar o percurso utilizando índices numéricos.
 - III. O uso da função embutida len(M) em uma matriz quadrada $N \times N$ retorna a quantidade total de células ($N \times N$).

É correto o que se afirma em:

- A) Apenas II.
- B) Apenas I e II.
- C) Apenas I e III.
- D) Apenas II e III.
- E) I, II e III.

39. Considere a operação de desempacotamento feita na linha 5:

```
1 def extrair_data(data_string):  
2     partes = data_string.split('/')  
3     return partes[0], partes[1], partes[2]  
4  
5 dia, mes, ano = extrair_data("15/08/1995")
```

Qual conceito possibilita essa operação?

- A) Assinatura com argumentos opcionais (Default Parameters).
 - B) Modularidade por Passagem de Referência Global.
 - C) Retorno Múltiplo de Valores, devolvidos na forma de uma Tupla implícita.
 - D) Estrutura de Condição de Parada Recursiva.
 - E) Atribuição estática de Escopo.
40. Considere um programa desenvolvido para percorrer uma lista de notas e identificar a maior nota registrada:

```
1 notas = [7.5, 8.0, 6.5, 9.2, 8.7]  
2  
3 maior = notas[0]  
4 posicao = 0  
5  
6 for i in range(len(notas)):  
7     if notas[i] > maior:  
8         maior = notas[i]  
9         posicao = i  
10  
11 print(posicao)
```

Qual a saída deste bloco de código?

- A) 9.2
- B) 3
- C) 4
- D) 8.7
- E) O código lançará um erro, pois listas não podem ser acessadas por índice.

41. Considere um programa desenvolvido para percorrer uma lista de tuplas, em que cada tupla armazena o nome de um produto e sua quantidade em estoque:

```
1 estoque = [  
2     ("Caneta", 15),  
3     ("Caderno", 8),  
4     ("Borracha", 22),  
5     ("Lapis", 5)  
6 ]  
7  
8 produto_maior = ""  
9 maior_quantidade = 0  
10  
11 for produto, quantidade in estoque:  
12     if quantidade > maior_quantidade:  
13         maior_quantidade = quantidade  
14         produto_maior = produto  
15  
16 print(produto_maior)
```

Qual a saída deste bloco de código?

- A) 22
 - B) Borracha
 - C) ("Borracha", 22)
 - D) Caderno
 - E) O código lançará um erro, pois tuplas não podem ser percorridas em um laço for.
42. Considere um programa desenvolvido para remover valores repetidos de uma lista, utilizando a conversão para conjunto:

```
1 numeros = [4, 2, 4, 8, 2, 10, 8]  
2  
3 sem_repeticao = set(numeros)  
4 resultado = list(sem_repeticao)  
5  
6 print(len(resultado))
```

Qual a saída deste bloco de código?

- A) 7
- B) 4
- C) 3

D) [4, 2, 8, 10]

E) O código lançará um erro, pois listas não podem ser convertidas em conjuntos.